STUDY NOTES

Practical Data Processing Python · Pandas · SQL

A structured recap of string parsing, missing-value handling, conditional feature engineering, and subquery patterns for data engineering & ML interviews.

Python Fundamentals | Pandas | SQL Subqueries | Interview Patterns

Data Engineering / ML Track

3 Sections

SECTION 1

🐍 Python — Practical Data Processing

Combines string methods with loops and exception handling to parse, clean, validate, and process structured records.

Important Pattern

```
for employee in employees:
    try:
        name, skills, salary = employee.split("|")
        name = name.strip().title()
        skills = skills.strip().split(",")
        salary = int(salary.strip())
        if salary < 0:
            raise ValueError("Salary cannot be negative.")
        print(f"{name} -> {skills} -> {salary}")
    except ValueError as e:
        print(f"Error: {e}")
```

Concepts Used

- `split()` → parse structured strings
- `strip()` → clean whitespace
- `title()` → normalize names
- `int()` → convert string to number
- `try/except` → handle bad records
- `raise ValueError` → validate data
- `for` loop → process records
- `f-string` → format output

🔥 Real-World Pattern

Particularly relevant to Data Engineering / ML pipelines:

> `Raw data → Parse → Clean → Validate → Handle bad records → Process valid records`

SECTION 2

🐼 Pandas

fillna()

Used to handle missing values.

```
df["salary"] = df["salary"].fillna(0)
```

> **Important distinction**
> `df.fillna(0)` returns a modified DataFrame but doesn't necessarily update your original variable unless you assign it.

Conditional Columns — np.select()

Creates a new column based on multiple conditions.

```
conditions = [
    df["salary"] >= 90000,
    df["salary"] >= 70000
]
choices = [
    "High",
    "Medium"
]
df["salary_category"] = np.select(
    conditions,
    choices,
    default="Low"
)
```

```
condition 1 → choice 1  |  condition 2 → choice 2  |  otherwise → default
```

Very useful for feature engineering and data categorization.

groupby()

```
avg_salary = df.groupby("department")["salary"].mean()
```

```
group employees by department → select salary → calculate average
```

General pattern:

```
df.groupby("column")["numeric_column"].mean()
```

Other Common Aggregations

- `.sum()`
- `.mean()`
- `.max()`
- `.min()`
- `.count()`

SECTION 3

🗃 SQL — Subqueries & Aggregation

Five interview-style questions solved below.

Q1 — Greater Than Company Average

```
SELECT name
FROM Employee_2
WHERE salary > (
    SELECT AVG(salary)
    FROM Employee_2
);
```

> `row value > overall aggregate`

Q2 — Greater Than Department Average

```
SELECT name
FROM Employee_2 e
WHERE e.salary > (
    SELECT AVG(salary)
    FROM Employee_2 e2
    WHERE e2.departmentId = e.departmentId
);
```

> ⭐ **Important — Correlated Subquery**
> The inner query depends on the current row of the outer query.

> `Outer employee → Find their department → Calculate that department's average → Compare employee salary`

Q3 — Highest-Paid Employee per Department

```
SELECT name, salary, departmentId
FROM Employee_2 e
WHERE e.salary = (
    SELECT MAX(salary)
    FROM Employee_2 e2
    WHERE e2.departmentId = e.departmentId
);
```

> `MAX() + correlated subquery`

Useful interview pattern for finding the highest/maximum value within each group.

Q4 — Second-Highest Distinct Salary

```
SELECT MAX(salary)
FROM Employee_2
WHERE salary < (
    SELECT MAX(salary)
    FROM Employee_2
);
```

> 🧠 **Remember**
> `MAX(value) WHERE value < MAX(value)` — finds the maximum salary below the highest salary.

Q5 — Departments with Average Salary > 75000

```
SELECT departmentId,
       AVG(salary) AS average_salary
FROM Employee_2
GROUP BY departmentId
HAVING AVG(salary) > 75000;
```

> ⭐ **Important Distinction**
> `WHERE` filters rows before grouping — `HAVING` filters groups after aggregation. Therefore `HAVING AVG(salary) > 75000` is appropriate.

RECAP

🎯 Today's Interview Patterns

Patterns you should now recognize immediately:

Pattern	Tool
Overall average comparison	Subquery
Group-specific average comparison	Correlated subquery
Maximum per group	Correlated subquery + MAX()
Second highest	MAX() + subquery
Filter aggregate groups	GROUP BY + HAVING
Missing values	fillna() / COALESCE()
Conditional column	np.select()
Group statistics	groupby()